

Python Cheatsheet — 1-Page Reference

Print it. Fold it. Walk in confident.

Section A — 12 Most-Asked Built-ins + Tricks

1. `zip(*iters)` — Parallel Iteration

Walks N iterables in lockstep; stops on shortest. Use `strict=True` (3.10+) to assert equal length.

```
for name, score in zip(names, scores, strict=True): ...
```

2. `enumerate(it, start=0)` — Index + Value

Use this instead of `for i in range(len(xs))`. Interview tell: range-len is junior.

```
for i, x in enumerate(orders, start=1): print(i, x)
```

3. `any()` / `all()` — Short-Circuit Truthiness

Both lazy. `any([])` is `False`; `all([])` is `True` (vacuous truth — a classic gotcha).

```
if any(o.status == "failed" for o in orders): retry()
```

4. `sorted(it, key=..., reverse=False)` — Stable Sort

Timsort, $O(n \log n)$, stable. `key=` runs once per element (cached). Use `operator.itemgetter` for speed.

```
sorted(users, key=lambda u: (-u.score, u.name)) # score desc, name asc
```

5. `filter(pred, it)` / `map(fn, it)` — Lazy Iterators

Return iterators in Py3, not lists. A comprehension is usually clearer (and same speed).

```
evens = (x for x in nums if x % 2 == 0) # prefer comp over filter
```

6. `itertools.chain` / `groupby` / `accumulate` — Iterator Toolkit

`groupby` only groups **consecutive** equal keys — sort first. `accumulate` does running totals.

```
from itertools import chain, groupby, accumulate
list(chain([1,2], [3,4])) # [1,2,3,4]
[(k, list(g)) for k, g in groupby(sorted(rows), key=lambda r: r.city)]
list(accumulate([1,2,3,4])) # [1,3,6,10]
```

7. `functools.reduce` / `partial` / `cache` — Composition + Memo

`@cache` is the unbounded `@lru_cache(maxsize=None)` (3.9+). Use on pure functions only.

```
from functools import reduce, partial, cache
add5 = partial(lambda x, y: x + y, 5) # add5(3) -> 8
@cache
def fib(n): return n if n < 2 else fib(n-1) + fib(n-2)
```

8. `collections.Counter` / `defaultdict` / `deque` — Daily-Driver Structures

`Counter.most_common(k)` is $O(n \log k)$. `deque` gives $O(1)$ both ends. `defaultdict` skips the `if key not in d` dance.

```
from collections import Counter, defaultdict, deque
Counter("mississippi").most_common(2) # [('i',4), ('s',4)]
buckets = defaultdict(list); buckets[k].append(v)
dq = deque(maxlen=100) # bounded ring buffer
```

9. `dict.get(k, default)` / `dict.setdefault(k, default)` — Safe Access

`get` reads with a fallback. `setdefault` reads-or-inserts in one step; returns the stored value.

```
count = totals.get(user_id, 0)
buckets.setdefault(city, []).append(order) # one lookup, not two
```

10. `pathlib.Path` — Modern Path Handling

Replaces `os.path`. Operator `/` joins. `Path.read_text()` / `.write_text()` skip the `open` boilerplate.

```
from pathlib import Path
cfg = Path.home() / ".myapp" / "config.json"
data = cfg.read_text(encoding="utf-8")
```

11. `@dataclass` — Auto `__init__` / `__repr__` / `__eq__`

Use `frozen=True` for hashable + immutable. Use `field(default_factory=list)` — never `[]` (trap #1).

```
from dataclasses import dataclass, field
@dataclass(frozen=True, slots=True)
class Order: id: int; tags: list[str] = field(default_factory=list)
```

12. `typing.Protocol` — Structural Subtyping

Duck typing the type-checker can verify. Object matches if it has the methods — no inheritance needed.

```
from typing import Protocol
class Closeable(Protocol):
    def close(self) -> None: ...
def shutdown(x: Closeable) -> None: x.close()
```

Section B — 8 Traps You Must Not Fall For

1. Mutable Default Arguments

`def f(x=[])` — the list is built **once at def-time**, shared across calls. **Say this:** `def f(x=None): x = [] if x is None else x`

2. Late-Binding Closures

`fns = [lambda: i for i in range(3)]` — all three return `2`, not `0/1/2`. Closure captures the **name**, not the value. **Say this:**

`fns = [lambda i=i: i for i in range(3)]` (bind via default arg)

3. `is` vs `==`

`==` checks **value**; `is` checks **identity**. `is None` / `is True` / `is False` are correct; `== None` is wrong. **Say this:** `if x is None:` — not `if x == None:`

4. Integer Caching (CPython detail)

`a = 256; b = 256; a is b` → `True`. At `257` → `False`. CPython interns `-5` to `256`. Never rely on it. **Say this:** Use `==` for equality, always. Mention "implementation detail" — interviewers love that phrase.

5. Float Math

`0.1 + 0.2 == 0.3` → `False` (IEEE 754). `round(0.5)` → `0` (banker's rounding, not 1). **Say this:** `math.isclose(a, b)` for approximate equality; `decimal.Decimal` for money.

6. List Aliasing on `[a] * n`

`grid = [[]] * 3` makes 3 references to **the same list**. `grid[0].append(1)` mutates all three. **Say this:** `[[] for _ in range(3)]` — fresh list per slot.

7. Iterator Exhaustion

A generator can only be consumed once. `zip(a, b)` exhausts `a` if `b` is shorter (3.10 fixes with `strict=True`). **Say this:** `list(gen)` if you'll iterate twice; or rebuild the generator.

8. Class Attribute Mutation

`class C: items = []` — `items` lives on the class, shared across instances. Mutating from `self.items.append(...)` leaks. **Say this:** Move mutable state into `__init__`: `def __init__(self): self.items = []`.

Section C — Time-Complexity of Common Operations

Operation	list	dict	set	tuple	deque	heapq (on list)
Index / access <code>x[i]</code>	O(1)	O(1) avg	n/a	O(1)	O(n) middle	O(1) <code>[0]</code> peek
Append / push (right)	O(1) amortized	O(1) avg <code>d[k]=v</code>	O(1) avg <code>s.add()</code>	n/a (immutable)	O(1) <code>appendright</code>	O(log n) <code>heappush</code>
Pop (right)	O(1)	O(1) avg <code>d.pop(k)</code>	O(1) avg <code>s.pop()</code>	n/a	O(1) <code>pop</code>	n/a
Pop-left / dequeue	O(n) <code>pop(0)</code>	n/a	n/a	n/a	O(1) <code>popleft</code>	O(log n) <code>heappop</code> (min)
Insert middle	O(n) <code>insert(i, x)</code>	n/a	n/a	n/a	O(n)	n/a
Membership (<code>x in c</code>)	O(n)	O(1) avg	O(1) avg	O(n)	O(n)	O(n)
Search by value	O(n) <code>.index()</code>	O(1) avg via key	O(1) avg	O(n)	O(n)	O(n)
min / max	O(n)	O(n) over keys	O(n)	O(n)	O(n)	O(1) for min (heap top)
Sort	O(n log n) Timsort	n/a (sort <code>.items()</code>)	n/a (sort to list)	n/a	O(n log n) via list	n/a (heap is partial order)

Worst cases: `dict` / `set` lookups degrade to O(n) on pathological hash collisions — rare in practice. **Pick-the-structure:** queue → `deque` . Priority queue → `heapq` . Membership-heavy → `set` . Insertion order matters → `dict` (3.7+ guarantees).

Section D — pandas Quick-Reference

groupby + agg (the one they ask)

```
df.groupby("city").agg(total=("amount", "sum"), avg=("amount", "mean"), n=("id", "count"))
df.groupby(["city", "tier"], as_index=False).agg({"amount": "sum"}) # flat result
```

groupby + apply (and why it's the perf cliff)

`.apply` runs your Python function **per group** — falls out of C-land. 10-100x slower than `.agg` . Use only when no built-in agg fits.

```
df.groupby("user").apply(lambda g: g.assign(rank=g["score"].rank())) # slow path
```

merge / join (the validate= you should know)

```
df1.merge(df2, on="user_id", how="left", validate="m:1", suffixes=("_a", "_b"))
# validate options: "1:1" | "1:m" | "m:1" | "m:m" — raises if assumption breaks
# how: "left" | "right" | "inner" | "outer" | "cross"
```

pivot vs pivot_table

	pivot	pivot_table
Duplicate (index, col) pairs	Raises <code>ValueError</code>	Aggregates (default <code>mean</code>)
Use when	Data already unique	You need aggregation

```
df.pivot(index="date", columns="city", values="sales")
df.pivot_table(index="date", columns="city", values="sales", aggfunc="sum", fill_value=0)
```

Vectorized vs apply (100x rule)

```
df["total"] = df["price"] * df["qty"] # vectorized — fast
df["total"] = df.apply(lambda r: r["price"] * r["qty"], 1) # apply — slow, avoid
```

Rule: if you can express it with `+` `-` `*` `/` `&` `|` `.where()` `.clip()` `.str.*` `.dt.*` , never `.apply` .

pandas 2.x nullable dtypes

`pd.NA` is a single, type-stable missing marker. `np.nan` is a float, so it forces float dtype on int columns.

```
s = pd.Series([1, 2, None], dtype="Int64")    # capital I – nullable integer
s.isna()                                     # True for pd.NA
df.convert_dtypes()                          # auto-upgrade to nullable dtypes
```

Trap: `pd.NA == pd.NA` → `pd.NA` (not `True`); `np.nan == np.nan` → `False`. Use `.isna()`, always.

Copy-on-write (pandas 2.x default in 3.0)

```
pd.set_option("mode.copy_on_write", True)    # opt in now; default soon
# Chained assignment (df[df.a > 0]["b"] = 1) silently no-ops under CoW. Use .loc.
```